

Welcome Offer

Access to everything. Now 30% off. [Upgrade now](#)

# useCallback Explained: Why React Keeps Creating New Functions



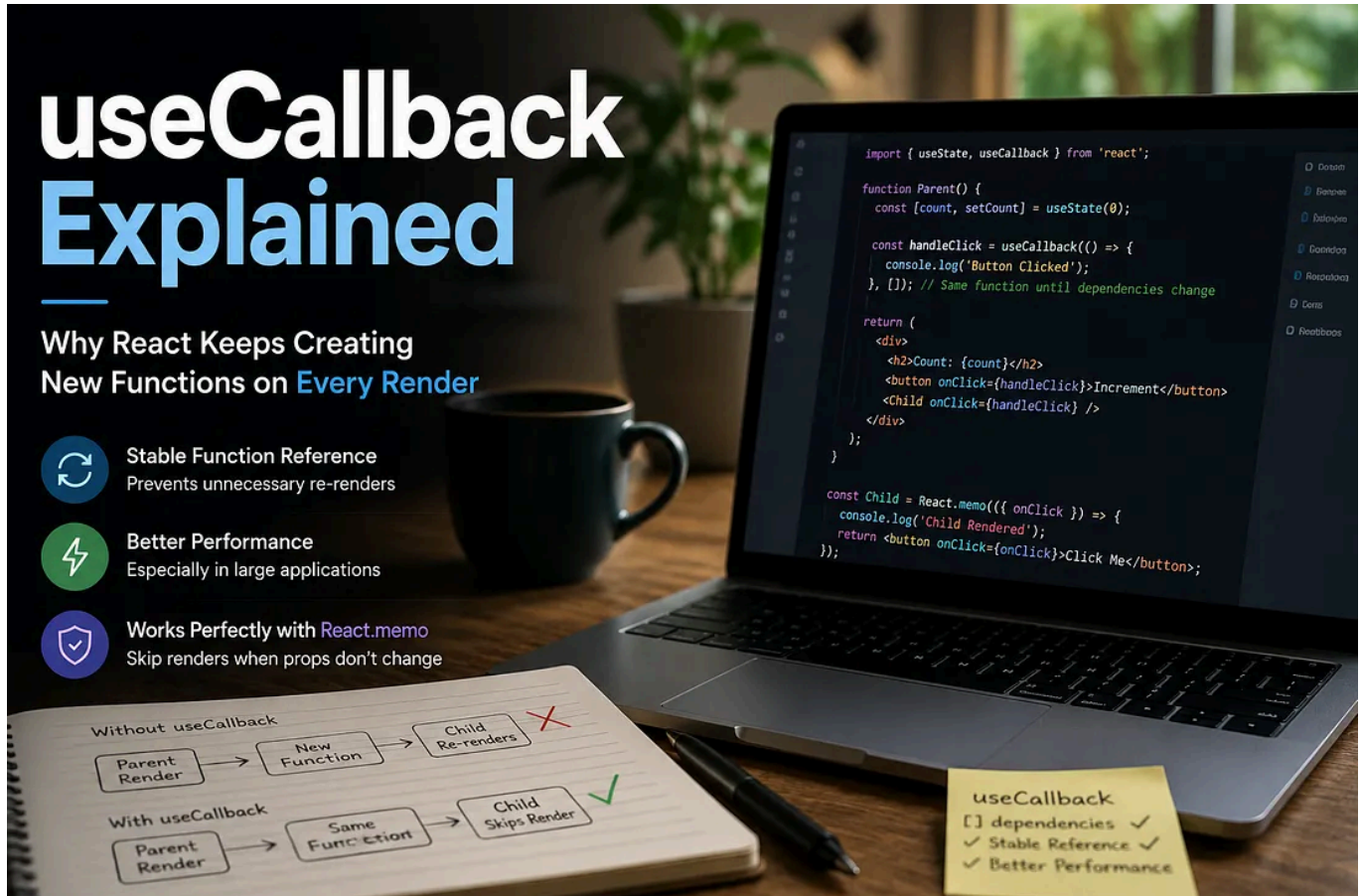
Devendra Dhote

Follow

3 min read · 2 days ago



4



After learning `React.memo`, many developers notice something strange.

They wrap a child component with `React.memo`, expecting unnecessary renders to stop.

But the child still renders.

The first question that comes to mind is:

“I didn’t change the data. Why is the component still rendering?”

The answer is surprisingly simple.

Because JavaScript keeps creating **new functions**.

Let’s understand why.

. . .

## The Real Problem

Imagine you have a parent component.

```
function Parent() {  
  return <Child onClick={() => console.log("Hello")} />;  
}
```

Every time the parent renders,

this function is created again.

```
() => console.log("Hello")
```

Even though the code looks identical,

JavaScript creates a **brand-new function object** on every render.

So internally, it looks like this:

Render 1

↓

Function A

Render 2

↓

Function B

Function A and Function B may do the same thing,

but they are **different references**.

For JavaScript,

```
Function A !== Function B
```

. . .

## Why Does This Matter?

Now imagine the child component is wrapped with `React.memo`.

```
const Child = React.memo(function Child({ onClick }) {  
  return <button onClick={onClick}>Click</button>;  
});
```

React compares the previous props with the new props.

The `onClick` function has a different reference every render.

So React thinks:

Props Changed

↓

Render Again

Even though nothing actually changed.

. . .

## The Solution: useCallback

`useCallback` tells React:

“Don’t create a new function unless its dependencies change.”

```
const handleClick = useCallback(() => {  
  console.log("Hello");  
}, []);
```

Now every render reuses the same function reference.

The flow becomes:

Parent Render

↓

Dependencies Changed?

↓

No

↓

Reuse Previous Function 

Instead of creating a brand-new function every time.

. . .

## Think of It Like a Phone Number

Imagine your friend calls you every day.

If your phone number changes every morning,

your friend has to save a new contact every day.



That's unnecessary work.

A better approach is keeping the same phone number until it actually needs to change.

That's exactly what `useCallback` does.

It keeps the same function reference until one of its dependencies changes.

. . .

## When Should You Use `useCallback`?

`useCallback` is useful when:

- Passing callback functions to memoized child components.
- Preventing unnecessary re-renders with `React.memo`.
- Providing stable callbacks to custom Hooks.
- Avoiding unnecessary effect executions when a function is a dependency.

These are the situations where function identity actually matters.

. . .

## When Should You Avoid It?

One of the biggest mistakes beginners make is wrapping every function with `useCallback`.

For example:

```
const sayHello = useCallback(() => {  
  console.log("Hello");  
}, []);
```

If this function is only used inside the same component,

there is usually no performance benefit.

React can simply create a new function.

Creating functions in JavaScript is already very fast.

. . .

## The Hidden Drawbacks of `useCallback`

Like every optimization, `useCallback` also has a cost.

### 1. Dependency Tracking

React has to store the callback and compare its dependencies on every render.

If the function is simple, this overhead may provide no real benefit.

. . .

### 2. More Complex Code

Compare these two examples.

Simple:

```
const handleClick = () => {  
  console.log("Clicked");  
};
```

With unnecessary optimization:

```
const handleClick = useCallback(() => {  
  console.log("Clicked");  
}, []);
```

The second version is longer and harder to read, without improving performance in most cases.

. . .

### 3. Stale Closures

If you forget a dependency,

your callback may capture old values.

```
const handleClick = useCallback(() => {  
  console.log(count);  
}, []);
```

Here, `count` will never update because it's missing from the dependency array.

Incorrect dependencies can introduce subtle bugs that are difficult to debug.

. . .

### 4. Optimizing Without Measuring

Many developers believe:

“More `useCallback` means better performance.”

Not necessarily.

If no component depends on a stable function reference, `useCallback` doesn't solve any real problem.

Always optimize after identifying a bottleneck — not before.

. . .

## **useCallback vs useMemo**

These two Hooks are closely related but solve different problems.

|                                     |                        |                                     |                                   |
|-------------------------------------|------------------------|-------------------------------------|-----------------------------------|
| <code>useCallback</code>            | <code>useMemo</code>   | Memoizes a function                 | Memoizes a calculated value       |
| Returns a stable function reference | Returns a cached value | Useful with <code>React.memo</code> | Useful for expensive calculations |

An easy way to remember them is:

`useMemo` remembers values. `useCallback` remembers functions.

. . .

## **Final Thoughts**

`useCallback` doesn't make your application automatically faster.

Its purpose is much more specific:

**Keep function references stable so React doesn't perform unnecessary work.**

Use it only when a stable callback actually matters.

Otherwise, let JavaScript create a new function.

Modern JavaScript engines are extremely efficient, and in many cases, creating a new function is cheaper than maintaining unnecessary memoization.

The best optimization isn't using more Hooks.

It's understanding **why** a Hook exists and using it only when it solves a real problem.

Happy coding ❤️

React ✓

JavaScript ✓

Optimization +

Programming ✓



**Written by Devendra Dhote**

350 followers · 8 following

Follow

MERN Stack Developer & Instructor. Writing about JavaScript, React, Next.js, backend development, and the engineering concepts behind modern web applications.